# Домашнее задание:
# RAG и протоколы взаимодействия AI-агентов

Разработка и внедрение AI-агентов · МФТИ

## Цель и результат

Подготовьте собственную базу знаний и подключите поиск по ней к агенту через **готовый MCP-сервер**. По запросу агент вызывает MCP-инструмент и возвращает top-k фрагментов с идентификаторами документа и фрагмента, а также источником.

```
документы → фрагменты → метаданные → индекс
          → MCP-инструмент поиска → агент → top-k и источники
```

Работа объединяет подготовку и проверку RAG-поиска с подключением внешнего инструмента по MCP. Развёрнутый ответ модели поверх найденных фрагментов - дополнительная часть по желанию.

## Что требуется

• Выбрать готовый MCP-сервер для векторной базы, базы знаний или поискового движка.

• Подготовить корпус, разбить документы, сохранить метаданные и построить индекс.

• Показать прямой вызов поиска через MCP и вызов того же инструмента агентом.

• Возвращать текст фрагментов, document_id, chunk_id и source.

• Проверить поиск на 15-20 запросах и дать краткую ручную оценку.

• Приложить код, конфигурацию и инструкцию воспроизведения.

Используйте готовый MCP-сервер. Если он предоставляет только поиск, подготовку и индексацию можно выполнить через SDK, CLI или API хранилища.

# 1. Выбор готового MCP-сервера

Выберите готовый сервер, который предоставляет поиск по вашим документам и возвращает метаданные фрагментов. Ниже перечислены несколько вариантов; можно использовать другой сервер с такими же возможностями.

| Вариант | Как использовать |
|---|---|
| Qdrant | Локальная база или отдельный сервис. На странице 4 приведены команды для версии сервера с поддержкой MCP 2026-07-28. Инструменты qdrant-store и qdrant-find сохраняют и находят текст с метаданными. |
| Chroma | Локальное хранилище фрагментов и эмбеддингов. Используйте официальный chroma-core/chroma-mcp. Версия 0.2.6 требует MCP SDK 1.6.0; запускайте сервер в отдельном окружении от клиента занятия. |
| Milvus | Поиск и операции с коллекциями через mcp-server-milvus. Нужен запущенный Milvus. Укажите поля фрагмента и метаданных, которые сервер должен возвращать в результатах. |
| Weaviate | Встроенный MCP по адресу /v1/mcp. В самостоятельном развёртывании включите MCP_SERVER_ENABLED и настройте права доступа. Для поиска используется weaviate-query-hybrid. |

Клиент MCP в режиме auto согласует редакцию протокола с сервером. Для серверов предыдущих редакций может использоваться initialize. Укажите согласованную редакцию и версии зависимостей в инструкции запуска.

## Что проверить у выбранного сервера

• Как называется инструмент поиска и какие аргументы принимает: query, limit/k, фильтры и имя коллекции.

• Где лежат метаданные в ответе. Идентификатор объекта базы не заменяет сохранённые document_id и chunk_id.

• Как задаётся k и возвращает ли сервер score, distance или только порядок результатов.

Облачный бесплатный тариф допустим. Ключи храните в .env; в сдачу включайте только .env.example.

## Документация

Qdrant: сервер и команды запуска

Chroma: MCP-сервер

Milvus: MCP-сервер

Weaviate: настройка встроенного MCP

LangChain: MCPAdapter

MCP: спецификация

# 2-4. Корпус, индекс и MCP-подключение

## 2. Подготовьте корпус документов

Минимальный объём: 5-10 документов, либо один большой документ с достаточным количеством смысловых фрагментов, либо эквивалентный текстовый корпус по выбранной теме.

Для каждого фрагмента сохраните **document_id**, **chunk_id** и **source** (путь, URL или другое описание источника). Желательны title, section, page или другой указатель.

```
metadata = {
    "document_id": "doc_01",
    "chunk_id": "doc_01_chunk_04",
    "source": "docs/risk_policy.md",
    "section": "Риски"
}
```

## 3. Выполните разбиение и индексацию

• Покажите загрузку исходных документов и правила разбиения на фрагменты.

• Присвойте стабильные идентификаторы и добавьте метаданные.

• Загрузите фрагменты в хранилище; проверьте, что поиск возвращает текст вместе с метаданными.

Если индексируете через SDK, согласуйте с MCP-сервером коллекцию, поля payload, имя и размерность вектора, а также модель эмбеддингов. Одна коллекция должна использовать совместимые эмбеддинги документов и запросов.

В Qdrant можно передавать подготовленные фрагменты в qdrant-store: текст в information, идентификаторы и источник в metadata. Для SDK-индексации сверьтесь с форматом официального сервера: payload содержит document и metadata; вектор именованный.

## 4. Подключите MCP-сервер

Приложите конфигурацию и команду запуска, название поискового инструмента, один прямой вызов и его ответ или журнал. В отчёте укажите версии сервера и клиента, транспорт и фактически согласованную редакцию MCP.

Для локального Qdrant сначала завершите процесс индексатора, если он открывал ту же папку базы. Локальное хранилище нельзя одновременно открыть из двух процессов. Альтернатива - отдельный Qdrant-сервис.

# Локальный запуск Qdrant

Для запуска готового официального сервера нужны Git и uv. Команды выполняются из корня вашего проекта; синтаксис подходит для терминала macOS/Linux и PowerShell.

```
git clone https://github.com/qdrant/mcp-server-qdrant.git vendor/qdrant-mcp
git -C vendor/qdrant-mcp checkout 9cfd5f1584e4de45517c335c2bb69f7a53e44055
uv sync --project vendor/qdrant-mcp --frozen --no-dev
```

Команды устанавливают исходник версии 0.9.0 с поддержкой MCP 2026-07-28. Пакет mcp-server-qdrant 0.8.1 из PyPI использует предыдущий стек и не является заменой этой конфигурации.

Окружение сервера находится в vendor/qdrant-mcp/.venv, отдельно от окружения агента. Зависимости клиента установите командой uv sync из материалов занятия. Серверные зависимости устанавливаются командой uv sync --project, приведённой выше.

## Конфигурация дочернего процесса

В Python из корня проекта соберите конфигурацию MCPAdapter. Переменные из env передаются только серверу:

```
from pathlib import Path

config = {
    "mcpServers": {
        "knowledge_base": {
            "command": "uv",
            "args": [
                "run", "--project", "vendor/qdrant-mcp",
                "--frozen", "--no-dev", "mcp-server-qdrant",
            ],
            "env": {
                "QDRANT_LOCAL_PATH": str(
                    Path(".runtime/knowledge-base").resolve()
                ),
                "COLLECTION_NAME": "knowledge_base",
                "QDRANT_SEARCH_LIMIT": "3",
            },
        }
    }
}
```

При первом запуске FastEmbed скачивает модель эмбеддингов; нужен интернет. По умолчанию сервер использует sentence-transformers/all-MiniLM-L6-v2. Для русскоязычного корпуса выберите подходящую поддерживаемую модель и задайте EMBEDDING_MODEL до индексации. Смену модели выполняйте с пересозданием индекса.

qdrant-find принимает query; число результатов задаёт QDRANT_SEARCH_LIMIT. В этом примере k=3. Если выбранный сервер передаёт k через конфигурацию, укажите это в отчёте; если через аргумент инструмента - передайте аргумент.

# 5. Подключение MCP-инструмента к агенту

Подключите инструменты через **langchain.mcp.MCPAdapter** и **await adapter.list_tools()**. Соединение должно оставаться открытым во время выполнения агента.

```
from langchain.mcp import MCPAdapter
from deepagents import create_deep_agent
from lab import make_model

model = make_model()
try:
    async with MCPAdapter(config) as adapter:
        tools = await adapter.list_tools()
        print([(t.name, t.description) for t in tools])
        search_tool = next(
            t for t in tools if t.name.endswith("qdrant-find")
        )
        agent = create_deep_agent(
            model=model,
            tools=tools,
            system_prompt=(
                f"Ищи в базе только через {search_tool.name}. "
                "Вызови его для запроса пользователя. "
                "Верни top-3 фрагмента, document_id, chunk_id "
                "и source из результата поиска."
            ),
        )
        result = await agent.ainvoke({
            "messages": [("user", "Найди фрагменты про риски ИИ")]
        })
        for message in result["messages"]:
            message.pretty_print()
finally:
    await model._client.aclose()
```

Пример выполняется в блокноте с общей настройкой занятия и заполненным .env. Можно использовать LangChain или LangGraph: условие задания - вызов MCP-поиска агентом. Наличие SDK-клиента хранилища без MCP-вызова этого условия не выполняет.

## Что должно быть видно в демонстрации

• Агент получил запрос, выбрал инструмент поиска и передал ему аргументы.

• В истории есть фактический результат MCP-инструмента с фрагментами и метаданными.

• Ответ агента содержит найденные фрагменты и их document_id, chunk_id, source.

У qdrant-find найденные записи упакованы в текст с тегами <entry>, <content> и <metadata>. Это формат ответа сервера, а не отдельные поля MCP-результата. При подготовке отчёта извлеките метаданные из каждой записи. Score этот инструмент не возвращает; используйте ранг.

# 6-7. Top-k и проверка качества поиска

## 6. Продемонстрируйте top-k поиск

Поиск принимает текст query и заданное k. Для каждого результата покажите текст фрагмента или отрывок, document_id, chunk_id, source, а также score, distance или rank. Оценку релевантности указывайте, если её возвращает инструмент; иначе фиксируйте порядок выдачи.

```
Query: Какие риски описаны в документе?
Top-k: 3

1. document_id=doc_01, chunk_id=doc_01_chunk_04, rank=1
   source=docs/risk_policy.md
   text=...
2. document_id=doc_03, chunk_id=doc_03_chunk_02, rank=2
   source=docs/ai_policy.md
   text=...
```

## 7. Проверьте поиск на 15-20 запросах

Запросы должны покрывать разные случаи:

• Точный факт и терминологический вопрос.

• Вопрос по конкретному документу и широкий смысловой вопрос.

• Вопрос без хорошего ответа в корпусе.

• Вопрос с несколькими релевантными источниками.

• Вопрос, где важны метаданные или источник.

Для каждого запроса сохраните результаты и краткую ручную оценку. Минимальные поля отчёта: query, top_k_results, document_id, chunk_id, score или rank, manual_judgement, comment.

| Запрос | Ожидаемый источник | Первый фрагмент | В top-3 | Комментарий |
|---|---|---|---|---|
| Что такое ...? | doc_03 | doc_03 / chunk_12 | да | Точное попадание |
| Какие риски ...? | doc_01, doc_04 | doc_04 / chunk_02 | да | Ответ в нескольких источниках |
| Кто автор ...? | нет ответа | doc_07 / chunk_01 | нет | Вопрос вне корпуса |

Идентификаторы выше - пример структуры отчёта. Для своей проверки укажите реальные документы, результаты и ожидаемые источники. Отрицательный запрос должен остаться в отчёте, даже если сервер возвращает несколько нерелевантных фрагментов.

# 8. Запуск, сдача и оценивание

## Инструкция воспроизведения

Опишите установку зависимостей, переменные окружения, подготовку или скачивание корпуса, разбиение и индексацию, запуск MCP-сервера, демонстрационный запрос и проверку на 15-20 запросах.

## Что сдавать

```
README.md
mcp_config.json / пример конфигурации
ingest.py или ingest.ipynb
agent_demo.py или agent_demo.ipynb
search_demo.py или search_demo.ipynb
eval_queries.csv или eval_report.md
requirements.txt или pyproject.toml + uv.lock
.env.example, если нужны переменные окружения
```

Ключи и учётные данные в сдачу не включайте. Если корпус нельзя распространять, укажите законный способ его получения, достаточный для воспроизведения.

## Критерии оценки

| Критерий | Вес |
|---|---|
| Корпус подготовлен: разбиение, метаданные, индексация | 20% |
| Подключён готовый MCP-сервер, понятно, какой инструмент используется | 20% |
| MCP-инструмент подключён к агенту на LangChain, LangGraph или Deep Agents | 20% |
| Агент возвращает top-k с document_id, chunk_id и метаданными источника | 15% |
| Проверка на 15-20 запросах с краткой ручной оценкой поиска | 15% |
| Воспроизводимая инструкция запуска | 10% |

## Дополнительно по желанию

Добавьте генерацию ответа со ссылками: query → MCP search → top-k → ответ модели. Основное задание остаётся поиском через MCP и проверкой его качества.

Перед сдачей проверьте: метаданные не потеряны, chunk_id стабильны, агент действительно вызывает MCP, в оценке есть отрицательные запросы, инструкция позволяет повторить запуск, секреты исключены из репозитория.